

Plan 003: Limpeza de artefatos mortos na raiz

Executor instructions: Follow this plan step by step. Run every verification command and confirm the expected result before moving to the next step. If anything in the “STOP conditions” section occurs, stop and report — do not improvise. When done, update the status row for this plan in `plans/README.md`.

Drift check (run first): `git diff --stat 94369a2..HEAD -- _test_db.py bench.bat bench.py perf_profile.py backup_db.py` Se qualquer arquivo mudou desde `94369a2`, PARE.

Status

- **Priority:** P2
- **Effort:** S
- **Risk:** LOW
- **Depends on:** none
- **Category:** tech-debt
- **Planned at:** commit `94369a2`, 2026-06-28

Why this matters

Quatro arquivos na raiz do projeto são artefatos mortos que não executam neste ambiente (Linux) e não são referenciados por nada. Um quinto (`backup_db.py`) duplica lógica de connection string. Removê-los reduz ruído e risco de drift de configuração.

Current state

- `_test_db.py` (68 linhas) — script de teste avulso com path Windows hardcoded: `sys.path.insert(0, r'c:\Users\Leonardo\OneDrive\Estudo\Python\...')`. Não executável neste ambiente.
- `bench.bat` (5 linhas) — batch Windows com path absoluto do usuário original. Não executável em Linux.
- `bench.py` (70 linhas) — benchmark de performance com conexão direta ao SQL Server (sem usar `SqlServerConnection`). Útil apenas com banco real.
- `perf_profile.py` (69 linhas) — profiling avulso de queries SQL + kaleido. Conexão direta via pyodbc, sem reusar as classes do projeto.
- `backup_db.py` (93 linhas) — script de backup funcional, mas duplica `_build_connection_string()` (linhas 17-28) que já existe em `src/infrastructure/sql/connection.py:30-43`. A diferença: `DATABASE=master` em vez de `DATABASE={settings.sql_database}`.

Commands you will need

Purpose	Command	Expected on success
Tests	<code>python -m pytest tests/unit -q</code>	12 passed
Verificar backup	<code>python -c "from src.infrastructure.sql.connection import SqlServerConnection; from src.config.settings import SETTINGS; c = SqlServerConnection(SETTINGS); print(c.build_connection_string()[:50])"</code>	<code>DRIVER={ODBC ...}</code>

Scope

In scope:

- `_test_db.py` — deletar
- `bench.bat` — deletar
- `bench.py` — deletar
- `perf_profile.py` — deletar
- `backup_db.py` — refatorar para reusar `SqlServerConnection`

Out of scope:

- `src/infrastructure/sql/connection.py` — não tocar (já tem o método)
- `perf_result.txt` — se existir, deletar também (gerado por [bench.py](#))

Git workflow

- Branch: `advisor/003-root-cleanup`
- Commits: um commit para deleções, um commit para refactor do `backup_db.py`
- Mensagens: `chore: remove scripts mortos na raiz do projeto` / `refactor: backup_db.py reusa SqlServerConnection.build_connection_string()`

Steps

Step 1: Deletar scripts mortos

```
rm _test_db.py bench.bat bench.py perf_profile.py
rm -f perf_result.txt # se existir
```

Verifique: `ls _test_db.py bench.bat bench.py perf_profile.py 2>&1` → todos reportam “No such file”

Step 2: Verificar que nada quebra

Verifique: `python -m pytest tests/unit -q` → 12 passed

Verifique: `grep -rn "_test_db\|bench\.bat\|bench\.py\|perf_profile" src/` → sem matches
(ninguém importa esses scripts)

Step 3: Refatorar `backup_db.py`

Substituir a função `_build_connection_string()` (linhas 17-28) para reusar `SqlServerConnection`:

```
# Antes (linhas 8-28):
import sys
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from src.config.settings import SETTINGS

def _build_connection_string() -> str:
    if SETTINGS.trusted_connection:
        return (
            f"DRIVER={{{{SETTINGS.sql_driver}}}};"
            f"SERVER={{SETTINGS.sql_server}};"
            f"DATABASE=master;"
            "Trusted_Connection=yes;"
        )
    return (
        f"DRIVER={{{{SETTINGS.sql_driver}}}};"
        f"SERVER={{SETTINGS.sql_server}};"
        "DATABASE=master;"
        f"UID={{SETTINGS.sql_username}};"
        f"PWD={{SETTINGS.sql_password}};"
    )
```

```
# Depois (no topo, substituindo sys.path + _build_connection_string):
from src.config.settings import SETTINGS
from src.infrastructure.sql.connection import SqlServerConnection

def _build_connection_string() -> str:
    """Reusa SqlServerConnection, forçando DATABASE=master para backup."""
    conn = SqlServerConnection(SETTINGS)
    cs = conn.build_connection_string()
    # Troca o database do settings por 'master' (backup roda em master)
    import re
    return re.sub(r"DATABASE=[^;]+", "DATABASE=master", cs)
```

Nota: a função `run_backup()` (linha 47) e todo o resto do arquivo permanecem inalterados.

Verifique: `python -c "from backup_db import _build_connection_string; s = _build_connection_string(); assert 'DATABASE=master' in s; print('OK')"` → OK

Step 4: Rodar suite completa

Verifique: `python -m pytest tests/ -q --tb=short` → todos os testes que passavam continuam passando.

Test plan

- `backup_db.py` não tem testes existentes. Após a refatoração, o código fica mais fácil de testar (usa `SqlServerConnection` que pode ser mockado).
- Nenhum teste novo necessário — `backup_db.py` é um script de CLI, não importado por nenhum código de produção.

Done criteria

- ☐ [] `_test_db.py`, `bench.bat`, `bench.py`, `perf_profile.py` deletados
- ☐ [] `perf_result.txt` deletado (se existia)
- ☐ [] `backup_db.py` não tem mais `_build_connection_string()` duplicada
- ☐ [] `python -m pytest tests/unit -q` → 12 passed
- ☐ [] `grep -rn "c:\\\\Users\\\\Leonardo" .` → sem matches (path Windows hardcoded)

STOP conditions

- Se `backup_db.py` tiver mudanças não documentadas neste plano, PARE e reporte o diff.
- Se algum dos scripts deletados for referenciado por `README.md` ou documentação, PARE e reporte.

Maintenance notes

- `backup_db.py` agora depende de `SqlServerConnection`. Se a conexão mudar (ex: driver ODBC), o backup segue automaticamente.
- Scripts de benchmark/profiling futuros devem ser colocados em `benchmarks/` (diretório que já existe no repositório).